Mode-Conditioned Advantage Normalization: A Length-Stratified Variant of GRPO for Multi-Mode Math Reasoning

Tue-Anh Vu
VinUniversity
`Programming Project Report - COMP2050 Artificial Intelligence`

June 22, 2026

Abstract

Group Relative Policy Optimization (GRPO) forms a group-relative advantage and, under the modern token-level (DAPO) loss, lets each rollout's gradient contribution grow with its length. We study a setting where this length effect turns harmful: *multi-mode* chain-of-thought fine-tunning, the setting introduced by our baseline project, in which a small 3B model is conditioned by a prompt prefix to reason at one of three depths (*low*/*medium*/*high*). The modes have systematically different completion lengths, so under the token-level - Decoupled Clip and Dynamic Sampling Policy Optimization (DAPO) loss, a verbose mode dominates each update for reasons unrelated to answer quality. In this individual project, I try to remove the imbalance directly inside the Reinforcement Learning (RL) stage. I propose **Mode-Conditioned GRPO (MC-GRPO)**, which multiplies the traditional GRPO group-normalized advantage by a per-mode completion-length factor. The results show that this makes every mode's expected per-sample gradient footprint equal, removing the between-mode length confound while preserving the within-mode, content-driven length signal that the DAPO loss legitimately rewards. On Llama-3.2-3B-Instruct, varient MC-GRPO with mode-matched batching reaches **80.5%** GSM8K 0-shot and **51.2%** MATH500 0-shot, versus **77.5%** and **41.6%** for traditional GRPO advantage under identical batching and setting, matching the baseline's result of 80.06% on GSM8K and 47.6% on MATH500 after GRPO stage.

1 Introduction

Large language models (LLMs) have become remarkably capable at multi-step reasoning, but that capability is concentrated in models with tens to hundreds of billions of parameters whose inference cost makes them impractical to deploy at scale. Knowledge distillation narrows this gap by transferring the reasoning ability of a large teacher into a small student: the teacher produces chain-of-thought (CoT) solutions, and the student is supervised-fine-tuned (SFT) to imitate them, recovering much of the teacher's competence at a fraction of the serving cost. Pure imitation, however, only matches the teacher's traces and never rewards the student for reaching a correct answer through its own reasoning. The now-standard recipe therefore follows SFT with a reinforcement-learning (RL) stage on verifiable rewards [4, 9], the distill-then-RL paradigm that underlies both our baseline project and this individual work.

Reinforcement learning from verifiable rewards has become the standard way to sharpen the mathematical reasoning of small language models. Building on Proximal Policy Optimization (PPO) [8], Group Relative Policy Optimization (GRPO) [9] drops the separate value model: for each question it samples a group of G rollouts, scores them with a reward, and forms a *group-relative* advantage by subtracting the group mean and dividing by the group standard deviation.

Under the modern token-level loss, Decoupled Clip and Dynamic Sampling Policy Optimization (DAPO) [12], each rollout's gradient contribution grows with its length, which is harmless for a single distribution, but harmful once a batch mixes reasoning styles of systematically different lengths.

Our previous project [10], considered as baseline, creates exactly such a setting. It is a reasoning distillation from a large teacher model, GPT-OSS-20B, which was utilized for its three reasoning effort modes (*low*/*medium*/*high*), into a small Llama-3.2-3B student model. We generated mode-specific CoTs and curated them by correctness and length filtering into clean three mode-reasoning instruction datasets for supervised fine-tuning (SFT). Following the usual distill-then-RL paradigm [4, 9], after the SFT initialization, we ran a GRPO training stage, with DAPO loss updates, on the *raw* GSM8K and MATH question pool *without* any mode conditioning. The baseline's best result, with SFT on the low-mode data followed by GRPO, is 80.06% GSM8K 0-shot and 47.6% MATH500 0-shot. However, this could be a misalignment. The SFT initialization is *mode-instructed* since it has learned prefix-conditioned reasoning style, yet the GRPO stage is *mode-blind*, optimizing on pure data. Besides, the overall baseline report paper also flags a limitation that mixing the reasoning modes causes training interference, which it attributes to their "mismatched token-length distributions."

In this personal project, I would try to make the GRPO stage *mode-aware*, allowing for the adaption with mode-instructed from SFT initialization in particular, and mixing reasoning modes training in general. Under the token-level loss, DAPO, which will be further discussed in §3, the high-reasoning mode may dominate each update because of its length set by prefix, not by answer quality. Therefore, I propose **Mode-Conditioned GRPO (MC-GRPO)**, which multiplies the group-normalized advantage by a per-mode completion-length factor, equalizing each mode's expected gradient footprint while preserving the within-mode, content-driven length signal the token-level loss legitimately uses.

Contributions.

1. **A mode-conditioned advantage estimator** (MC-GRPO) that rescales the group-relative advantage by a per-mode completion-length factor, with an analysis showing it removes the mode length confound that DAPO loss creates in a multi-mode batch setting, as discussed in (§3).
2. **An evaluation** on GSM8K and MATH500 showing MC-GRPO is favorable to traditional GRPO advantage with DAPO loss, while matching the reported baseline.

2 Related Work

From PPO to GRPO. PPO [8] optimizes a clipped surrogate objective with a learned value function providing the advantage baseline. GRPO [9] removes the value model, using the mean reward of a group of rollouts to the same question as the baseline, therefore cheaper and well matched to verifiable-reward math.

Token biases in GRPO, and how DAPO addresses them. Liu et al. [7] (Dr. GRPO) analyze two biases in the GRPO objective. The *response-level length bias* comes from the per-response $1/|o_i|$ token-average, which makes the per-token update larger for shorter responses, so the policy learns to be brief when correct and verbose when wrong. The *question-level difficulty bias* comes from the $1/\text{std}(\mathbf{R})$ factor, which upweights low-variance for too-easy or too-hard questions; Dr. GRPO note that advantage normalization is "typically computed across an entire batch," so the per-group version distorts weighting. REINFORCE++ [6] makes the same point about the variance term and advocates computing it over the global batch. DAPO [12], the recipe both the baseline and this project build on, responds to the length bias with a *token-level*

loss aggregation (sum over all tokens, divide by the batch token total) in place of the per-response average. DAPO itself observes that under this token-level loss "longer sequences can have more influence on the overall gradient update" [12, §3.3], treating it as desirable for long-CoTs as a longer correct chain has more tokens worth reinforcing. However, it is only true for a single mode distribution, and becomes a per-mode *confound* once a batch mixes modes of prefix-dictated lengths, and is exactly what MC-GRPO targets.

Multi-mode reasoning and the baseline setting. Controlling reasoning "effort" which is consistent with study from [4] and our baseline [10]. In the baseline, we distilled GPT-OSS-20B's math reasoning into a Llama-3.2-3B student by a curated mode-specific SFT data. The baseline then runs its GRPO stage on the two raw datasets GSM8K and MATH *without* mode conditioning. We hypothesize this is a missed opportunity: the model receives a *mode-instructed* SFT initialization, but is then optimized on unconditioned data, so the GRPO stage is misaligned with the SFT stage in both instruction and token length. Can a GRPO variant instead adapt to the SFT-initialized setting and allow for mixing modes training without inteference? This question leads me to MC-GRPO, which still trains on the raw GSM8K and MATH questions, but attaches the three mode tags and mode prefixes to each one. Both the baseline and this work use the DAPO loss, which already fixes the token-length bias of the original GRPO loss; MC-GRPO then adjusts the advantage inside the update by a per-mode length factor, so the optimizer is not biased by a response's length across modes while still keeping DAPO's token-level property (§3).

3 Method

3.1 Where length enters: from GRPO's token bias to a multi-mode confound

For a question q with G rollouts $o_1, \ldots, o_G$ and rewards $\mathbf{R} = \{R(q, o_1), \ldots, R(q, o_G)\}$, GRPO and DAPO use the *same* group-relative advantage, identical for every token of rollout i:

$$\hat{A}_i = \frac{R(q, o_i) - \text{mean}(\mathbf{R})}{\text{std}(\mathbf{R}) + \varepsilon}. \tag{1}$$

They differ only in how the clipped per-token surrogate $\ell_{i,t} = \min\big(\rho_{i,t}\hat{A}_i,\ \text{clip}(\rho_{i,t}, 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}})\hat{A}_i\big)$, with $\rho_{i,t} = \pi_\theta(o_{i,t} \mid q, o_{i,<t}) / \pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})$, is aggregated:

$$\text{GRPO (sample-level):} \quad \mathcal{J}_{\text{GRPO}} = \mathbb{E}\Big[\tfrac{1}{G}\textstyle\sum_i \frac{1}{|o_i|}\sum_t \ell_{i,t}\Big], \tag{2}$$

$$\text{DAPO (token-level):} \quad \mathcal{J}_{\text{DAPO}} = \mathbb{E}\Big[\frac{1}{\sum_j |o_j|}\textstyle\sum_i \sum_t \ell_{i,t}\Big]. \tag{3}$$

In the project, I use this DAPO loss for: token-level aggregation (3) with decoupled clipping (ε=0.2, $\varepsilon_{\text{high}}$=0.28), matching the baseline. I do not use DAPO's dynamic sampling or overlong-reward shaping.

Loss	per-token weight	aggregate per-rollout weight
GRPO (sample-level)	$\frac{1}{G\,\lvert o_i\rvert}$	$\frac{1}{G}$
DAPO (token-level)	$\frac{1}{\sum_j \lvert o_j\rvert}$	$\frac{\lvert o_i\rvert}{\sum_j \lvert o_j\rvert}$

Table 1: The two loss aggregations weight a single token, and a whole rollout.

Per-token vs. aggregate weight. Because $\hat{A}_i$ is constant across the $|o_i|$ tokens of rollout i, it is useful to separate the weight each loss puts on a *single token* from the *whole rollout* (the sum of tokens in that response). Liu et al. [7] show the per-response $1/|o_i|$ in (2) is itself a length bias, which the DAPO token-level form (3) removes. Table 1 works these out. GRPO's $1/|o_i|$ makes the *per-token* weight shrink with length, yet the *aggregate* per-rollout weight is $1/G$, length-independent. DAPO's token-level loss flips this: it removes the per-token bias when every token weighted equally, but the aggregate per-rollout weight becomes $\propto |o_i|$, as a new length of response bias. DAPO does not abolish length dependence, DAPO moves it from the per-token level in GRPO to the aggregate leve, where DAPO regards it as a feature that longer correct chains have more tokens worth reinforcing.

The aggregate dependence then becomes a mode confound. We use DAPO, so the aggregate weight of rollout i is $w_i \propto |o_i|$. Aggregating over a mode, let mode m contribute N_m rollouts with mean length $L_m = \mathbb{E}[\,|o_i| \mid m(i) = m\,]$, the total pull of mode m is

$$W_m \;=\; \sum_{i:\, m(i)=m} w_i \;\propto\; N_m\, L_m. \tag{4}$$

With an equal number of rollouts per mode, the modes are still weighted $\propto L_m$: **the verbose mode dominates each update by a factor set by its mean length**. In ordinary training this is a feature. In the multi mode setting, it is a *confound* as L_m is dictated by the mode prefix, not by answer quality. As a result, a mode is up-weighted merely for having been instructed to reason at length. This is the imbalance MC-GRPO removes.

3.2 MC-GRPO: mode-conditioned length normalization

MC-GRPO multiplies a single per-mode factor onto the advantage (1). With $m(i) \in \{\mathit{low}, \mathit{medium}, \mathit{high}\}$ the mode of sample i, L_m the mean completion length of mode m in the batch, and $\bar{L} = \frac{1}{|\mathcal{M}|}\sum_{m\in\mathcal{M}} L_m$ the mean of the three per-mode means:

$$\boxed{\hat{A}_i^{\text{MC-GRPO}} = \frac{R(q,o_i) - \text{mean}(\mathbf{R})}{\text{std}(\mathbf{R}) + \varepsilon} \;\cdot\; \frac{\bar{L}}{L_{m(i)} + \varepsilon}} \tag{5}$$

The factor is < 1 for above-average-length modes and > 1 for below-average ones, and equals 1 if a single mode is present. We estimate L_m on *mode-matched* batches: the dataset stores each question as the consecutive triple $(q, \mathit{low}), (q, \mathit{medium}), (q, \mathit{high})$, so a batch holds the same questions in all three modes and L_m reflects the mode effect with question difficulty held fixed. We use completion *length* as the per-mode statistic because it is the quantity that varies substantially across modes, shown in Table 4.

Why it equalizes modes. Substituting (5) into the mode pull (4), the $\bar{L}/L_m$ factor multiplies W_m:

$$W_m^{\text{MC-GRPO}} \;\propto\; N_m L_m \cdot \frac{\bar{L}}{L_m} \;=\; N_m\, \bar{L}.$$

The mode mean L_m *cancels*: every mode now contributes $\propto N_m$, i.e. equally when the batch holds equal rollouts per mode. Equivalently, the expected per-rollout weight inside any mode is $L_m \cdot (\bar{L}/L_m) = \bar{L}$, the same constant for all modes.

It complements the DAPO loss; it does not fight it. Decompose a rollout's length into its mode mean and a within-mode relative factor, $|o_i| = L_{m(i)} \cdot \big(|o_i|/L_{m(i)}\big)$. The DAPO weight $\propto |o_i|$ carries both; the MC-GRPO weight is

$$|o_i| \cdot \frac{\bar{L}}{L_{m(i)}} \;=\; \bar{L} \cdot \frac{|o_i|}{L_{m(i)}}.$$

The mode-mean $L_{m(i)}$ cancels into the constant $\bar{L}$, while the *within-mode* factor $|o_i|/L_{m(i)}$ survives. So MC-GRPO removes only the prefix-driven, between-mode length component and keeps the content-driven, within-mode length signal DAPO legitimately rewards. The two act on different parts of the same quantity, DAPO across tokens within a sequence, MC-GRPO across mode means, so they compose.

A concrete example. Suppose one update batch holds two *low* rollouts (length $|o|$=90 each) and two *high* rollouts ($|o|$=130 each), and that all four are correct, so $\hat{A}_i$=+1 for each. The batch totals $2{\cdot}90 + 2{\cdot}130 = 440$ tokens.

Under DAPO, a rollout's pull on the update is its token share $|o_i|/\sum_j |o_j|$: each low rollout contributes $90/440 \approx 20.5\%$ and each high rollout $130/440 \approx 29.5\%$. Summed per mode, *high* takes 59% of the update and *low* only 41%, a gap created purely by length, even though all four answers are equally correct.

Under MC-GRPO, the per-mode factor is $\bar{L}/L_m$ with $\bar{L} = (90 + 130)/2 = 110$: low is scaled *up* by $110/90 \approx 1.22$ and high *down* by $110/130 \approx 0.85$. A rollout's effective weight is then $|o_i| \cdot (\bar{L}/L_m)$: low $= 90 \cdot 1.22 = 110$ and high $= 130 \cdot 0.85 = 110$. All four now weigh the same 110, so each mode takes exactly 50%.

The within-mode signal survives. The factor uses the mode mean L_{high}=130, not each rollout's own length, so it is the same 0.85 for every high rollout. A high rollout of length 156 still gets $156 \cdot 0.85 \approx 133$, against $104 \cdot 0.85 \approx 88$. The longer, more-developed chain still pulls harder within the mode. MC-GRPO thus removes only the across-mode, while keeping the within-mode that "this rollout reasoned more" signal.

4 Experimental Setup

4.1 Data

The two stages use different data, both derived from the baseline's resources:

- **SFT stage.** We reuse the baseline's curated reasoning data, specifically D'_{low} 10,388 rows, the low-mode set distilled from GPT-OSS-20B and filtered by correctness and length filtering. This provides the mode-instructed initialization from which every GRPO run later starts on.
- **MC-GRPO stage.** Built from the *raw* GSM8K [1] and MATH [5] train splits, I attach all three mode prefixes, as well as a mode tag, to every question: 14,969 unique questions $\times$ 3 modes 44,907 rows **(multi-mode dataset)**. Each row is `{question_id, prompt (prefix+format), mode, answer}`. Regarding the GRPO stage, both in the baseline and this project, I use this raw (q, a) pool rather than the curated D' datasets, simply because GRPO is on-policy RL, in which the student generates its *own* reasoning and is rewarded only by final-answer correctness, so it needs just the question and its ground-truth answer, not the teacher traces that the SFT stage imitates.

4.2 Model and implementation

The student is Llama-3.2-3B-Instruct [3]. Experiments are conducted on a single NVIDIA RTX 3090 GPU. I first run SFT on the curated D'_{low} data, a single initialization shared by all GRPO conditions.

Regarding the MC-GRPO stage, the reward matches the baseline paper's specification, binary correctness $r_{\text{acc}} \in \{0, 1\}$ plus a 5-gram repetition penalty with floor -0.5, giving $r \in [-0.5, 1.0]$. I run on HuggingFace TRL's `GRPOTrainer` (v1.4.0) [11], starting from the SFT checkpoint above; traditional GRPO and MC-GRPO differ by exactly one thing in the optimizer, the remaining are inherited unchanged.

- **Traditional GRPO advantage**: I leave the optimizer untouched, keep GRPO's group-normalized advantage (1), `scale_rewards="group"`, and set `loss_type="dapo"` for DAPO's token-level loss and decoupled clipping.
- **MC-GRPO**: I subclass `GRPOTrainer` and override a single method, `_generate_and_score_completions`. After `super()` generates all rollouts and returns the dict with `"advantages"` (1) and `"completion_mask"`, the subclass reads per-mode mean lengths from the mask and multiplies $\bar{L}/L_m$ onto the advantages. The two statistics live at different scopes: the base advantage (1) is normalized within each prompt's G rollouts, whereas L_m is the mean over all rollouts of mode m *in the generation batch*, computed in the single `_generate_and_score_completions` call after all rollouts exist.

Training configuration. The SFT initialization stage is ran with LoRA r=16, α=32, lr 2×10^{-5}, and 1 epoch. Regarding the MC-GRPO stage, to match with the baseline paper's GRPO setup, Table 2 lists the configuration. One optimizer step processes a generation batch of 72 completions: with G=8 rollouts per prompt that is 9 prompts, and under mode-matched batching those 9 prompts are 3 questions $\times$ 3 modes. Each mode therefore contributes $3\times8=24$ rollouts per step, the sample on which L_m is estimated.

Knob	Value	Knob	Value
Loss	DAPO (token-level)	Rollouts G	8
$\varepsilon_{\text{low}}, \varepsilon_{\text{high}}$	0.2, 0.28	Sampling	T=1.0, top-p=0.95
β (KL)	0.0	Max completion	2048 token
`scale_rewards`	`"group"`	Learning rate	1×10^{-5}
Reward	binary + 5-gram pen. (-0.5)	Training steps	300
Prompts/step	9 (= 3 Q $\times$ 3 modes)	Warmup ratio	0.05
Per-device batch	1 $\times$ 72 grad-accum	Precision	bf16, grad-ckpt
Backbone init	SFT LoRA r=16	Generation	vLLM colocate

Table 2: MC-GRPO configuration table

Evaluation. I use `lm-eval-harness` v0.4.11 [2] with a vLLM backend. Because the model emits answers in `\boxed{...}` as format suffix prompt, two custom tasks are forked as `gsm8k_cot_boxed_Nshot` to match with. MATH500 uses `minerva_math500`. Each merged checkpoint is evaluated with the **model's default without mode prefix (no-prefix), to make it fair with universial studies.**

5 Results

Method	Training Data (GSM8K & MATH)	Loss	GSM8K		MATH500	
			0-shot (strict)	5-shot (strict)	0-shot (math-verify)	4-shot (math-verify)
GRPO advantage	Multi-mode	DAPO	77.45	76.57	41.6	43.0
MC-GRPO	Multi-mode		**80.52**	**78.24**	**51.2**	**43.0**
Baseline GRPO	*pure*		*80.06*	*71.95*	*47.60*	*45.20*

Table 3: Main results table

Main results. Table 3 compares the two algorithms trained on the identical multi-mode dataset against the reported baseline trained on the pure one. MC-GRPO with mode-matched batching reaches **80.52%** GSM8K 0-shot and **51.2%** MATH500 0-shot, against **77.45%** and **41.6%** for traditional GRPO advanatge under the same multi-mode training. The advantage also holds in the few-shot column, and relative to the baseline GRPO of 80.06% GSM8K 0-shot.

Discussion. The results have shown that GRPO advantage without length re-balancing in the multi-mode setting **underperforms** even the baseline's result with pure dataset. Trained on the multi-mode dataset, i.e. more conditioning signal, it scores 77.45% and 41.6%, below the baseline GRPO's 80.06% and 47.60%. Simply mixing modes therefore *harmful* despite the extra supervision. This can likely be attributed to the symptom predicted in §3.1: under the DAPO token-level loss and multi-mode reasoning setting, modes of prefix-dictated lengths pull the update $\propto N_m L_m$, so the verbose mode dominates each step for reasons unrelated to answer quality and the optimizer is driven by a length confound rather than by correctness. The drop is the optimization-stage echo of the "mismatched token-length distributions" interference our baseline flagged at the data stage. MC-GRPO differs from this by canceling between-mode length §3.2; leading to better evaluation results compared to traditional GRPO advantage.

6 Ablation study

6.1 Per-mode behaviour and the length signal

Mode	Mean resp. length
low	86.2
medium	114.0
high	103.2
no-prefix	101.4

Table 4: Response lengths under different mode prefixes

To check that the per-mode length signal MC-GRPO relies on is actually present, I further evaluate it on GSM8K with the three mode prefixes (*low*/*medium*/*high*). Each run records the mean completion length the prefix induces in Table 4. The table have shown that *low* generates the shortest completions 86.2, with *high* in between 103.2. This confirms that completion length carries a prefix-driven per-mode signal, the quantity MC-GRPO normalizes during training.

6.2 Ablation: mode-matched vs. random batching

MC-GRPO estimates L_m from the batch, so batch composition is an axis worth probing. The *matched* batches of my main setting hold the same questions in all three modes; a *random* batch estimates L_m across different questions per mode.

Setting	GSM8K 0-shot	GSM8K 5-shot
MC-GRPO, matched	**80.52**	**78.24**
MC-GRPO, random	79.61	77.10

Table 5: Batching ablation

Table 5 has shown better performance for *matched* batching. My hypothesis is that *matched* batches hold question difficulty fixed across the three modes, so the per-mode mean L_m that

MC-GRPO divides by reflects the pure mode-length effect rather than noise from comparing different difficulty-level questions, giving the normalizer a cleaner estimate.

7 Limitations

- **Computation budget is the binding constraint.** All experiments ran on rented GPU, a single RTX 3090 on vast.ai, where each GRPO run costs around 5-7 hours, on top of the trial-and-error needed to stabilize the pipeline. Therefore, every number reported is the only seed 42. Besides, all GRPO conditions start from a single SFT checkpoint trained on D'_{low} only; I could not afford to also SFT on D'_{medium}, D'_{high}, and 3 modes combination to check whether the MC-GRPO effect is robust to a different mode-instructed initialization starting point.
- **Narrow scope.** Results cover one backbone, Llama-3.2-3B-Instruct and two math benchmarks; whether mode-conditioned normalization transfers to other model and task families is untested.

8 Conclusion

I introduced MC-GRPO, a minimal mode-conditioned variant of GRPO that rescales the group-normalized advantage by a per-mode completion-length factor. An analysis of the token-level DAPO loss shows the variant removes the between-mode length confound the loss creates in a multi-mode batch, the optimization echo of the length-mismatch interference our baseline observed at the data stage, while preserving the within-mode length signal the loss legitimately uses. On a single seed, MC-GRPO with matched batching reaches 80.5% GSM8K 0-shot and 51.2% MATH500 0-shot, outperforming the traditional GRPO advantage in the relative setting.

References

[1] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. GSM8K dataset.

[2] Leo Gao, Jonathan Tow, Stella Biderman, et al. A framework for few-shot language model evaluation. `https://github.com/EleutherAI/lm-evaluation-harness`, 2023. lm-eval-harness; Zenodo, doi:10.5281/zenodo.10256836.

[3] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. Model: Llama-3.2-3B-Instruct.

[4] Q. He, S. Yuan, X. Li, et al. ThinkDial: An open recipe for controlling reasoning effort in large language models. *arXiv preprint arXiv:2508.18773*, 2025.

[5] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2021. arXiv:2103.03874.

[6] Jian Hu, Jason Klein Liu, Haotian Xu, and Wei Shen. REINFORCE++: Stabilizing critic-free policy optimization with global normalization. *arXiv preprint arXiv:2501.03262*, 2025.

[7] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-zero-like training: A critical perspective. In *Conference on Language Modeling (COLM)*, 2025. Dr. GRPO; arXiv:2503.20783.

[8] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[9] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. Introduces GRPO.

[10] Hai-Au Trinh, Tue-Anh Vu, Dai-Nhan Tran, Uyen Khoi-Minh Huynh, and Anh-Khoi Nguyen. Curating multi-mode CoT for efficient math reasoning with GPT-OSS, 2026. Accepted at ICISN 2026; to appear in Springer Lecture Notes in Networks and Systems (LNNS), Scopus-indexed. Paper: `https://drive.google.com/file/d/1h4tAQVgOE2j2mdRPCkVZfMi-7g2i32Ht/view`; Code: `https://github.com/Koii2k3/LLaMA-OSS`.

[11] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Quentin Gallouédec, et al. TRL: Transformer reinforcement learning. `https://github.com/huggingface/trl`, 2020.

[12] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025. Decoupled Clip and Dynamic sAmpling Policy Optimization.